NoSQL Systems

MongoDB

"A true NoSQL document-oriented database system"

Vinu Venugopal
ScaDS.ai Lab, IIIT Bangalore

MongoDB

- Short form of "**Humongous DB**"
- **Open-source tool** but not an Apache tool; MongoDB today is a company
- **Document oriented** database: Stores all its data in the form of documents
- Follows **flexible schema: True NoSQL DB**
- The data model is based on the **JSON** (JavaScript Object Notation) **format**
- Queries are written in **JavaScript like language**
- Has its **own replication schema** (instead of HDFS) and **integration with MapReduce** (instead of Hadoop).
- A MongoDB server can be provisioned directly on a cloud setup or downloaded as a binary for all OS environments: https://www.mongodb.com/download-center (~~V 6.0.5~~ **V 8.0** (current))

MongoDB Access Mode

- **Command-line shell**

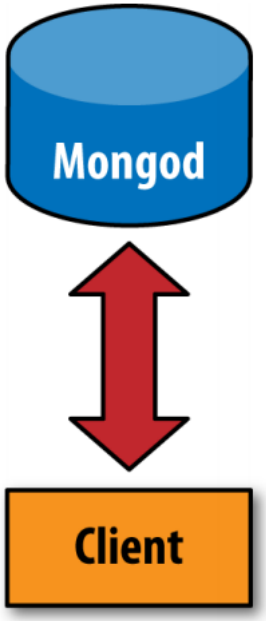
  % mongod (start a single server database)

  % **mongo**

  >

- **Other Access modes**

  JavaScript



- We first need to start the server using “mongod” it is a daemon process that would take care the requests from the client.

- And then, when you run the “mongo” **(mongosh)** command you would get an interactive shell.

MongoDB shell – basic commands

- To see the list of databases on the server:

```
>show dbs
```

- To display the database you are using:

```
> db
test
```
(default database)

- To switch to another database (or to create a new database):

```
> use <database_name>
```

```
% mongo <database_name> -u <name> -p <password>
```

MongoDB shell

- **The db.stats() method returns a document with statistics about the database system's state:**

```
> db.stats()
{
"db" : "test",
"collections" : 1,
"views" : 0,
"objects" : 49,
"avgObjSize" : 33,
"dataSize" : 1617,
"storageSize" : 36864,
"numExtents" : 0,
"indexes" : 1,
"indexSize" : 36864,
"scaleFactor" : 1,
"fsUsedSize" : 85887053824,
"fsTotalSize" : 499963174912,
"ok" : 1
}
```

Number of objects (specifically, documents) in the database across all collections.

Total size of the uncompressed data held in the database. The dataSize decreases when you remove documents.

Sum of the space allocated to all collections in the database for document storage, including free space.

Total number of indexes across all collections in the database.

Refer: https://docs.mongodb.com/manual/reference/command/dbStats/

MongoDB shell

SQL Terms/Concepts	MongoDB Terms/Concepts
database	database
table	collection
row	document
column	field
primary key	primary key
index	index

- To list the available collections:

```
>show collections
```

- To insert a single document into the collection ‘testCollection’:

```
>db.testCollection.insert({x:1})
```

  - If the collection name does not exist, it is created on-the-fly

MongoDB shell

- To insert multiple documents using **for** loop:

```
> for (var i=2; i<25; i++)  db.testCollection.insert({x:i})
```

- Unless specified explicitly, a document **automatically obtains a 12-byte object identifier**, which composed of a timestamp, the client machine ID, the client process ID, and a 3-byte incremented counter

  E.g., `"_id" : ObjectId("610c58131e829f5748ae541a")`

MongoDB shell

- To insert multiple documents using **for** loop:

```
> for (var i=2; i<25; i++)  db.testCollection.insert({x:i})
```

- Unless specified explicitly, a document **automatically obtains a 12-byte object identifier**, which composed of a timestamp, the client machine ID, the client process ID, and a 3-byte incremented counter

- To find all documents in the collection 'testCollection':

```
> db.testCollection.find()
```

- The find() function in fact returns a cursor to the documents contained in this collection. This cursor can be moved as an iterator over the objects in the collection
- E.g., To find all the documents with value '1' or '2' in their x field:

```
> db.testCollection.find({$or:[{x:1}, {x:2}]})
```

- To drop the entire collection:

```
> db.testCollection.drop()
```

Scripting

- Can write mongo scripts in JavaScript
- Opening a new connection:

```
conn = new Mongo();
db = conn.getDB("myDatabase");
```

- If not on default port:

```
db = connect("localhost:27020/myDatabase");
```

Shell Helper	JavaScript Equivalents
show dbs	db.adminCommand(‘listDatabases’)
use <db>	db = db.getSiblingDB(‘<db>’)
show collections	db.getCollectionNames()
show users	db.get Users()

Scripting

- Can write mongo scripts in JavaScript
- Opening a new connection:

```
conn = new Mongo();
db = conn.getDB("myDatabase");
```

- If not on default port:

```
db = connect("localhost:27020/myDatabase");
```

Shell Helper	JavaScript Equivalents
show dbs	db.adminCommand('listDatabases')
use <db>	db = db.getSiblingDB('<db>')
show collections	db.getCollectionNames()
show users	db.get Users()

- To execute a JavaScript file:

```
mongo localhost:27017/test myjsfile.js
```

- Use the `--eval` option to pass the mongo shell a JavaScript fragment:

```
mongo test --eval "printjson(db.getCollectionNames())"
```

MongoDB Data Model

Flexible Schema

- "**Collections**" (tables in MongoDB) **do not** enforce a schema
- "**Documents**" within a collection **can have** different fields

Data model is based on JSON Documents/Objects

MongoDB Data Model

Flexible Schema

- "**Collections**" (tables in MongoDB) **do not** enforce a schema
- "**Documents**" within a collection **can have** different fields

Data model is based on JSON Documents/Objects

Example:

```
{
    _id: ObjectId("5099803df3f4948bd2f98391"),
    name: { first: "Alan", last: "Turing" },
    birth: new Date('Jun 23, 1912'),
    death: new Date('Jun 07, 1954'),
    contribs: [ "Turing machine", "Turing test" ],
    }
```

- String values are generally double-quoted
- Arguments of the function calls are single-quoted

- { … } indicates a map of key-value pairs;
- […] indicates an array of values under a given key;
- … these may be nested up to any level.

- Internally stored in a binary format, called **BSON** (Binary JSON)

- MongoDB currently support 19 data types defined for BSON

MongoDB Data Model

How can we maintain References among Documents? – Similar to FKs in RDBMS

Embedded Data Models

Normalized Data Models

MongoDB Data Model

References among Documents

Embedded Data Models

- **Related data** is stored in a **single document** structure.
- **Denormalized** data model (Downside? Redundancy)
- Embedding document structures in a field or array within a document.

```
{
  _id: <ObjectId1>,
  username: "123xyz",
  contact: {
              phone: "123-456-7890",
              email: "xyz@example.com"
            },
  access: {
             level: 5,
             group: "dev"
           }
}
```

Embedded sub-document

Embedded sub-document

- For many use cases in MongoDB, the denormalized data model is optimal.

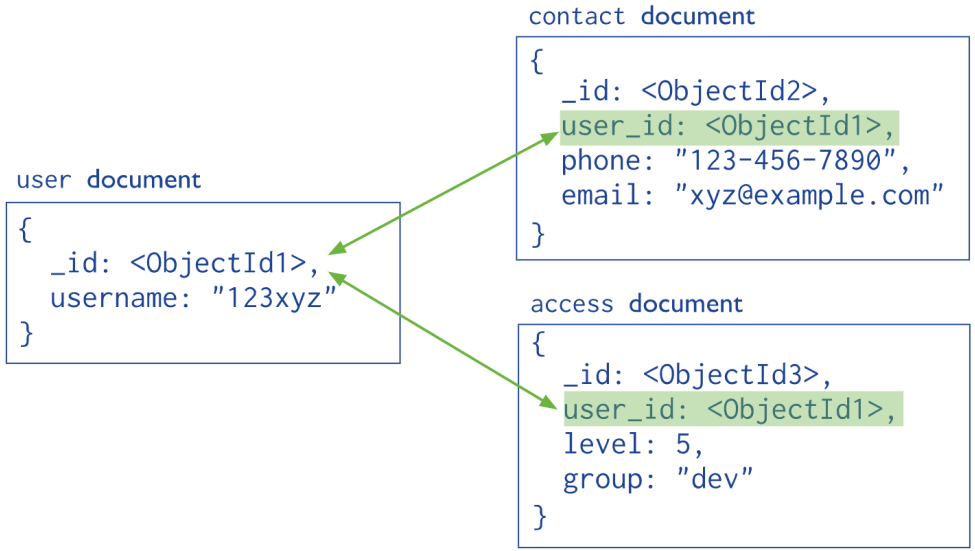
MongoDB Data Model

References among Documents

Normalized Data Models

- Stores the relationships between data **by including links or references** from one document to another

- Application must **resolve these references** to access the related data



- Less redundancy for one-to many relationships!

MongoDB Data Model

Modeling One-to-Many Relationships – Multiple options are there!

<u>Option 1:</u> Embedded Documents ("Denormalized Schema")

```
{
    title: "MongoDB: The Definitive Guide",
    author: [ "Kristina Chodorow", "Mike Dirolf" ],
    published_date: ISODate("2010-09-24"),
    pages: 216,
    language: "English",
    publisher: { name: "O'Reilly Media", founded: 1980, location: "CA" }
 }

{
    title: "50 Tips and Tricks for the MongoDB Developer",
    author: "Kristina Chodorow",
    published_date: ISODate("2011-05-06"),
    pages: 68,
    language: "English",
    publisher: { name: "O'Reilly Media", founded: 1980, location: "CA" }
}
```

MongoDB Data Model

Modeling One-to-Many Relationships – Multiple options are there!

Option 1: Embedded Documents ("Denormalized Schema")

```
{
    title: "MongoDB: The Definitive Guide",
    author: [ "Kristina Chodorow", "Mike Dirolf" ],
    published_date: ISODate("2010-09-24"),
    pages: 216,
    language: "English",
    publisher: { name: "O'Reilly Media", founded: 1980, location: "CA" }
 }

{
    title: "50 Tips and Tricks for the MongoDB Developer",
    author: "Kristina Chodorow",
    published_date: ISODate("2011-05-06"),
    pages: 68,
    language: "English",
    publisher: { name: "O'Reilly Media", founded: 1980, location: "CA" }
}
```

Embedding publisher details in the book documents will result in **redundancies**.

MongoDB Data Model

Modeling One-to-Many Relationships

Option 2: Separate Documents with a **Mutable Array.** (This can avoid redundancies.)

```
{ name: "O'Reilly Media",
  founded: 1980,
  location: "CA", books: [12346789, 234567890, ...]
}

{ _id: 123456789,
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216, language: "English" }

{ _id: 234567890,
  title: "50 Tips and Tricks for the MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English" }
```

MongoDB Data Model

Modeling One-to-Many Relationships

Option 2: Separate Documents with a **Mutable Array.** (This can avoid redundancies.)

```
{ name: "O'Reilly Media",
  founded: 1980,
  location: "CA", books: [12346789, 234567890, ...]
}

{ _id: 123456789,
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216, language: "English" }

{ _id: 234567890,
  title: "50 Tips and Tricks for the MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English" }
```

What if there are too many books by a single publisher?

This will cause Growing Array Problem!

MongoDB Data Model

Modeling One-to-Many Relationships

Option 3: Avoiding Growing Array

```
{ _id: "oreilly",
  name: "O'Reilly Media",
  founded: 1980,
  location: "CA" }

{ _id: 123456789,
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216,
  language: "English", publisher_id: "oreilly" }

{ _id: 234567890,
  title: "50 Tips and Tricks for the MongoDB Developer",
  author: "Kristina Chodorow", publisher_id: "oreilly",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English" }
```

Keeping the reference of the publisher in each of its book document.

MongoDB Data Model

Comparison to Foreign Keys in SQL

- A foreign key in SQL also expresses a one-to-many relationship (however**, at the schema** rather than the instance level.)

```
CREATE TABLE Books (
    title CHAR(30)  PRIMARY KEY,
    author VARCHAR(255),
    published_by INT,
    FOREIGN KEY (published_by) REFERENCES Publisher(id) );
```

- Many-To-Many relationship can be captured using two foreign keys in SQL

“N” Books --- “M” Authors

MongoDB Data Model

Many-To-Many (N:M)

Option 1: Two Way Reference Embedding

Book

```
{ _id: 1,
title: "A tale of two people",
categories: ["drama"],
authors: [“A”, “B”] }

{ _id: 2,
title: "A tale of two space ships",
categories: ["scifi"],
authors: [“A”] }
```

Author

```
{ _id: “A”,
name: "Peter Standford",
books: [1, 2] }

{ _id: “B”,
name: "Georg Peterson",
books: [2] }
```

Embed the object id of author in the book document.

MongoDB Data Model

Many-To-Many (N:M)

Option 1: Two Way Reference Embedding

Book

```
{ _id: 1,
title: "A tale of two people",
categories: ["drama"],
authors: [“A”, “B”] }

{ _id: 2,
title: "A tale of two space ships",
categories: ["scifi"],
authors: [“A”] }
```

Author

```
{ _id: “A”,
name: "Peter Standford",
books: [1, 2] }

{ _id: “B”,
name: "Georg Peterson",
books: [2] }
```

Embed the object id of author in the book document and vice versa.

MongoDB Data Model

Many-To-Many (N:M)

Option 2: One Way Reference Embedding
Embed the reference only in one side of the relationship

Book

```
{ _id: 1,
title: "A tale of two people",
categories: [“a”],
authors: [“A”, “B”] }

{ _id: 2,
title: "A tale of two space ships",
categories: [“b"],
authors: [“A”] }
```

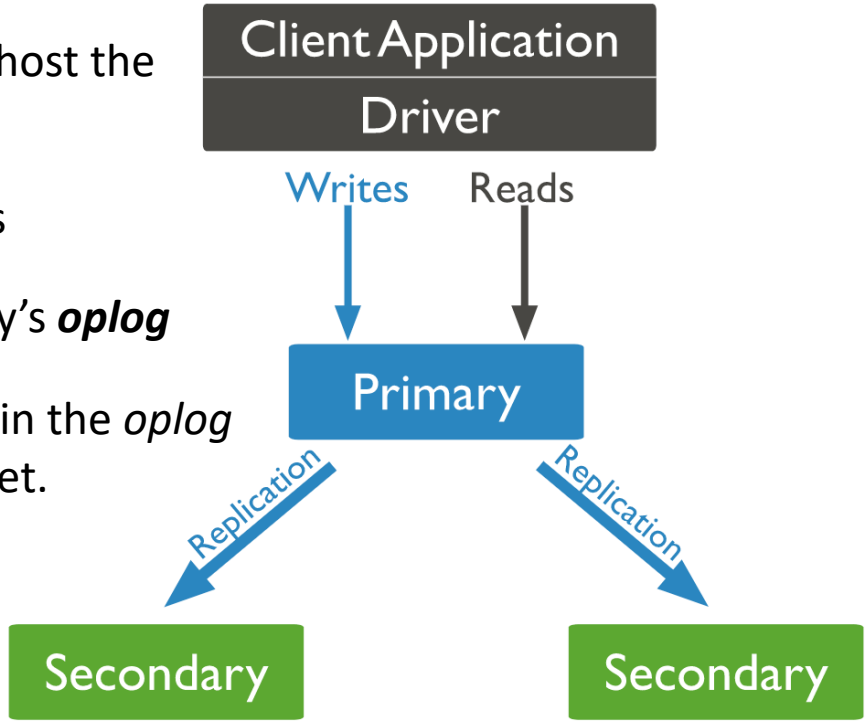
Catagories

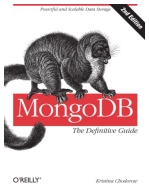
```
{ _id: “a”,
  name: "drama”,
  book:[1,…] }

{ _id: “b”,
  name: "scifi ”,
  book:[1,…] }
```

- There is **no silver bullet,** and you should always create the most appropriate data model that meets the needs of how your data will be queried.

Data Storage & Replication

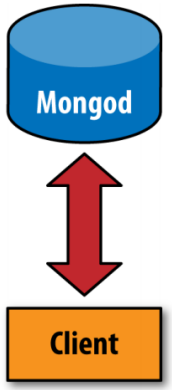
- Not integrated with Hadoop
- Provides its own replication scheme (so-called "**replica sets**")
- **Replica set:** group of **mongod** instances that host the same data set
- Primary mongod receives all write operations
- All secondary instances maintains the primary's ***oplog***
- Each secondary instance apply operations as in the *oplog* so that they always maintain the same data set.

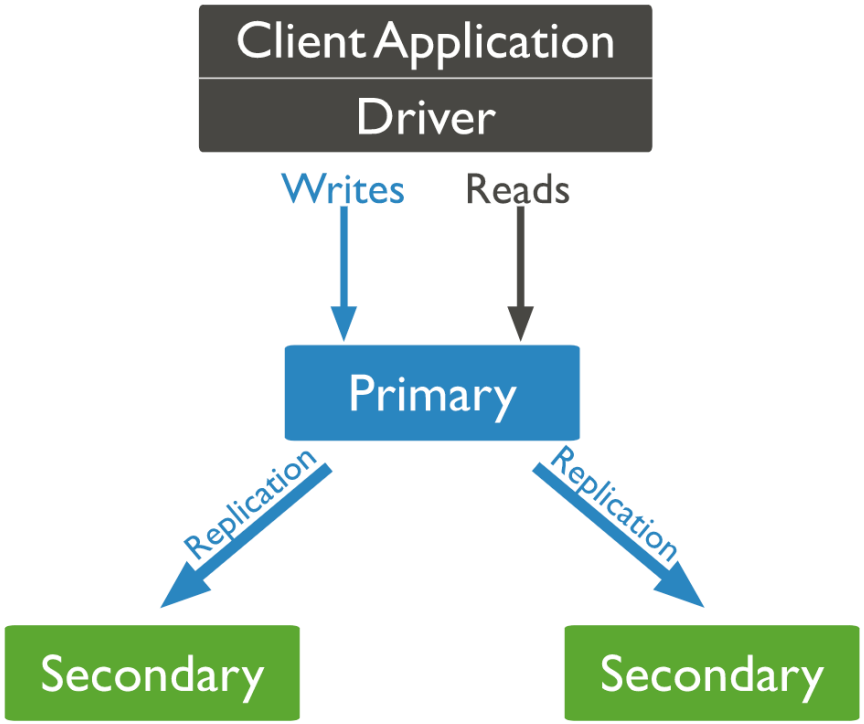




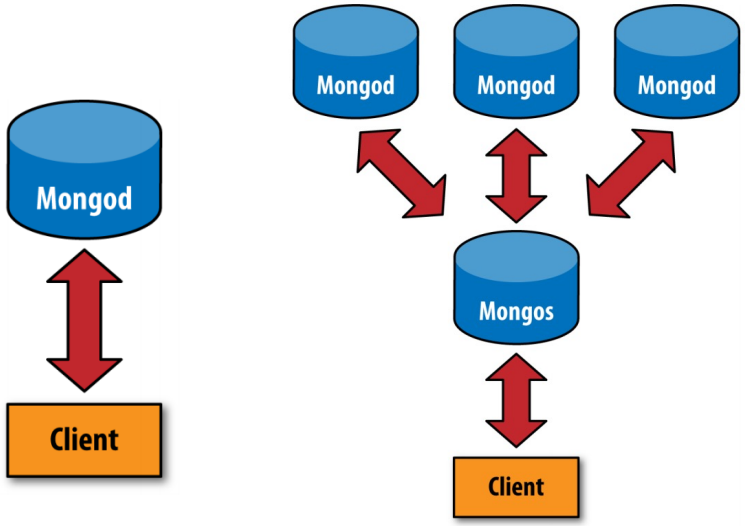
Reference: MongoDB: The Definitive Guide, Second Edition

Data Storage & Replication





Data Storage & Replication

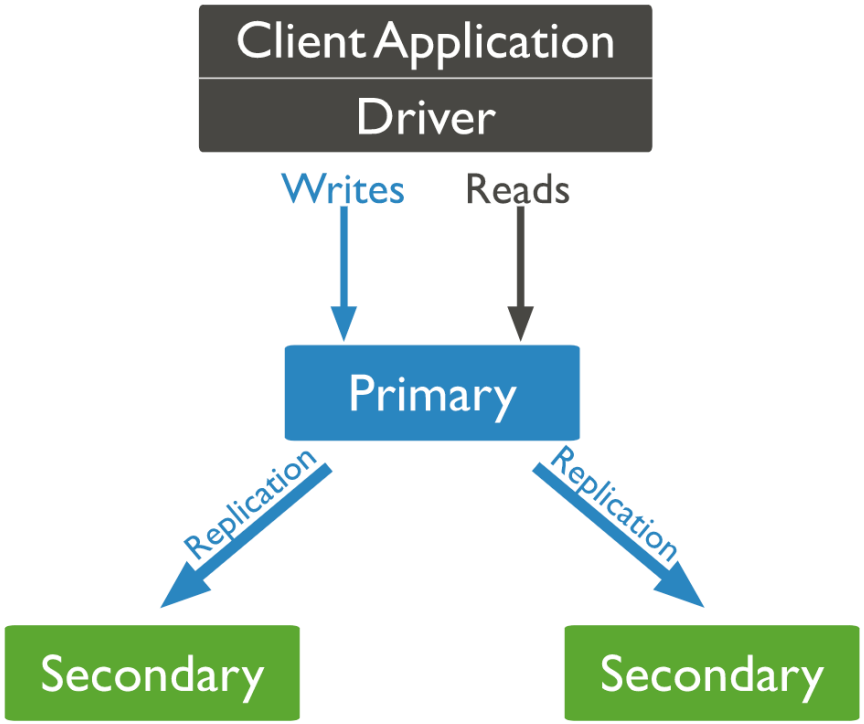


After employing the replication scheme.

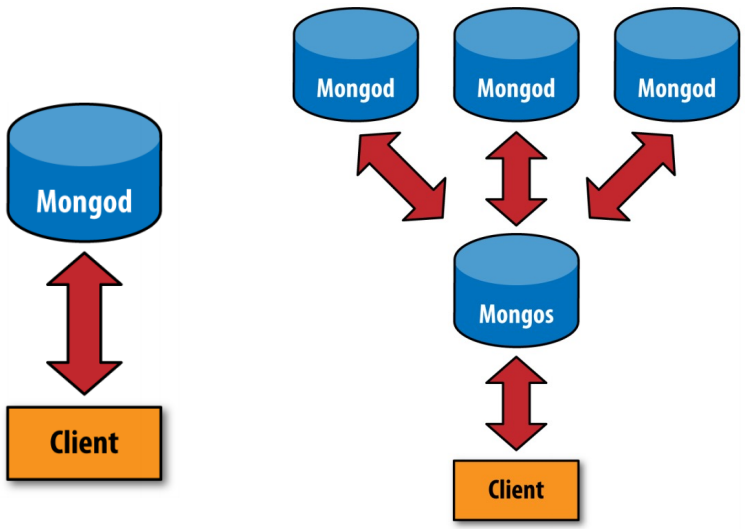
Note: One mongod instance **in a replica set** is "primary" and rest of all are 2ndary instances"

The primary instance would receive all write operations.

If the primary is not available then that replica set would elect a secondary to become a primary.

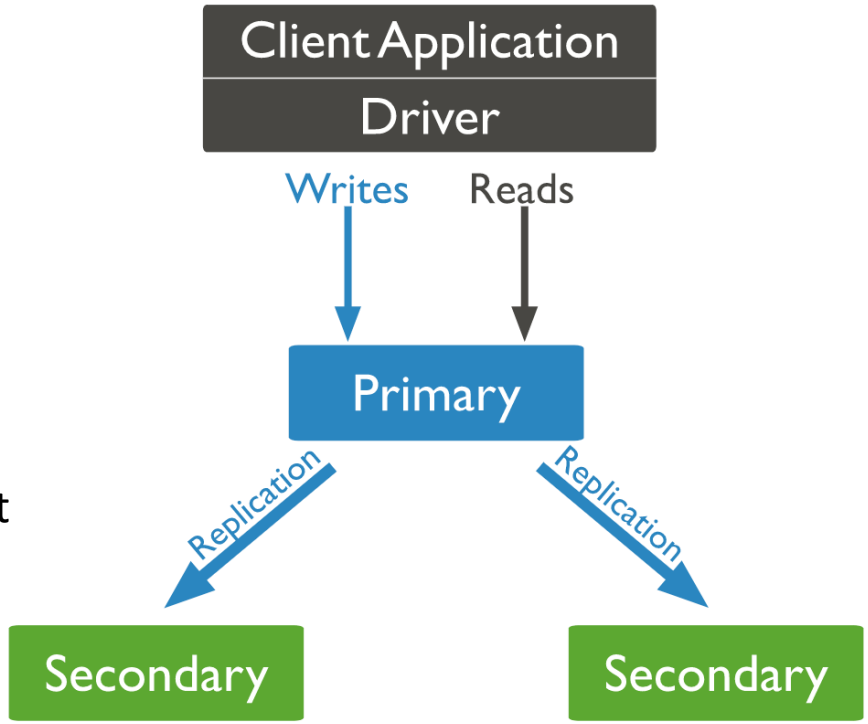


Data Storage & Replication



After employing the replication scheme.

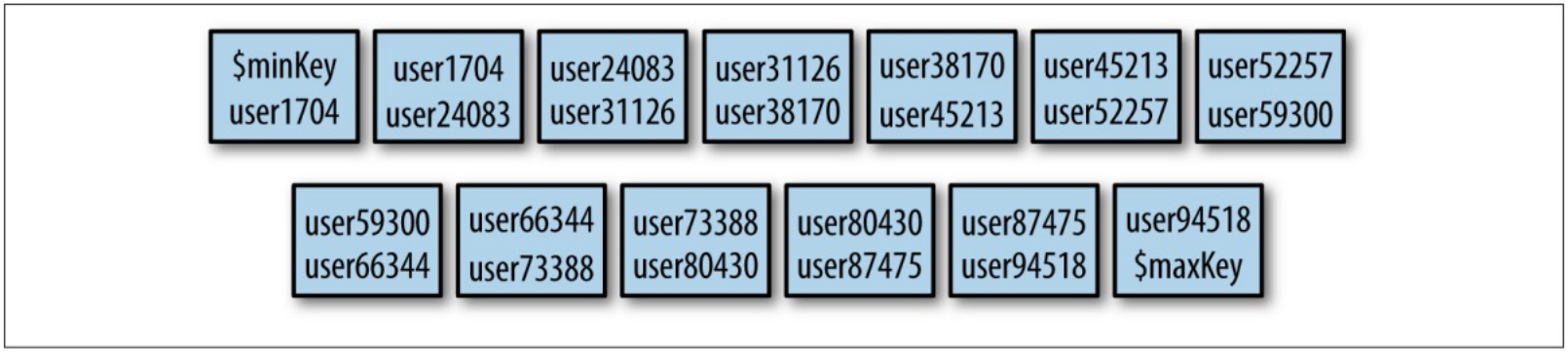
- When an operation is processed by a replica set primary, the effect of that operation must also be written to the database.
- And, the description of that operations must be written into oplog, so that the operations can be replicated to secondaries.

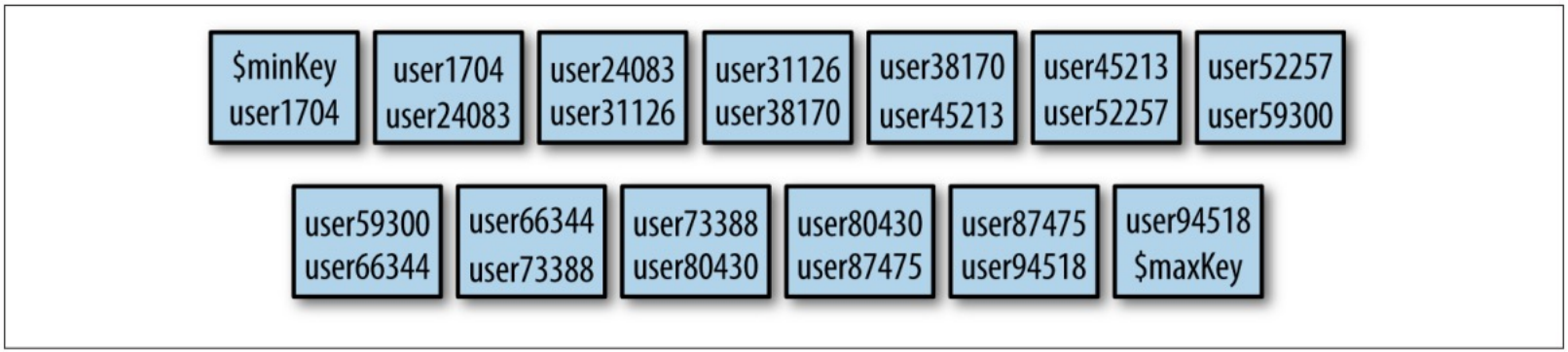


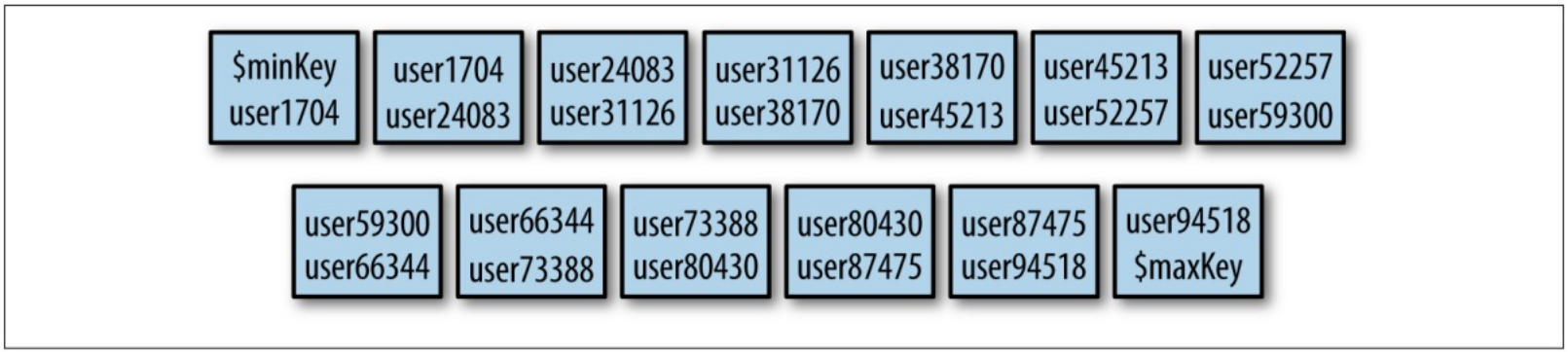
If it is a transaction: *locally committed* intially, and after propagating to enough nodes, *majority commit* would happen.

Sharding

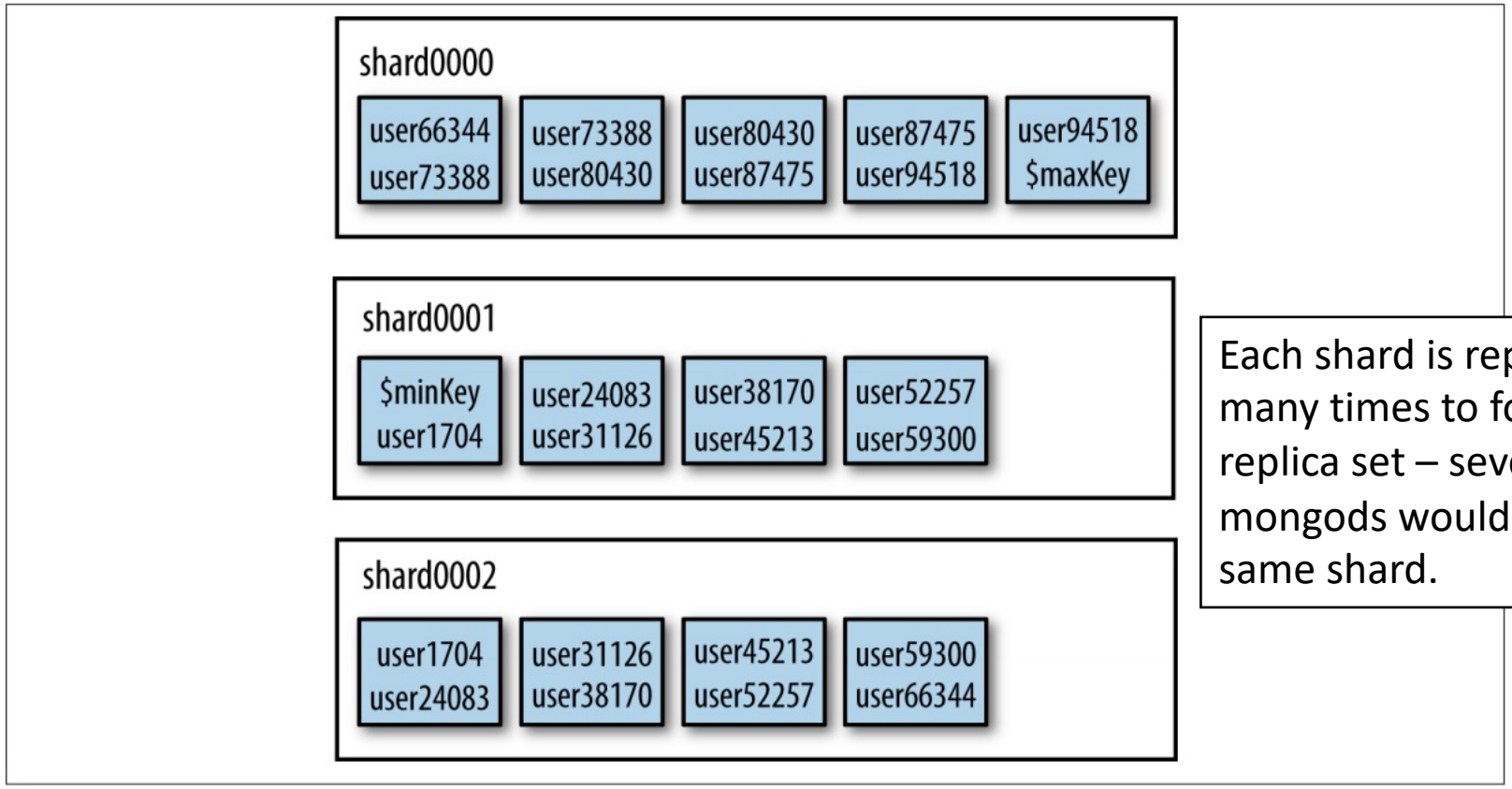
- Horizontally Partitioned Collections, based on a sharding key (E.g., userID)
- In contrast to replication, divides a data set and distributes the data over multiple MongoDB servers




$minKey
user1704
user1704
user24083
user31126
user31126
user38170
user38170
user45213
user45213
user52257
user52257
user59300
user59300
user66344
user66344
user73388
user73388
user80430
user80430
user87475
user87475
user94518
user94518
$maxKey

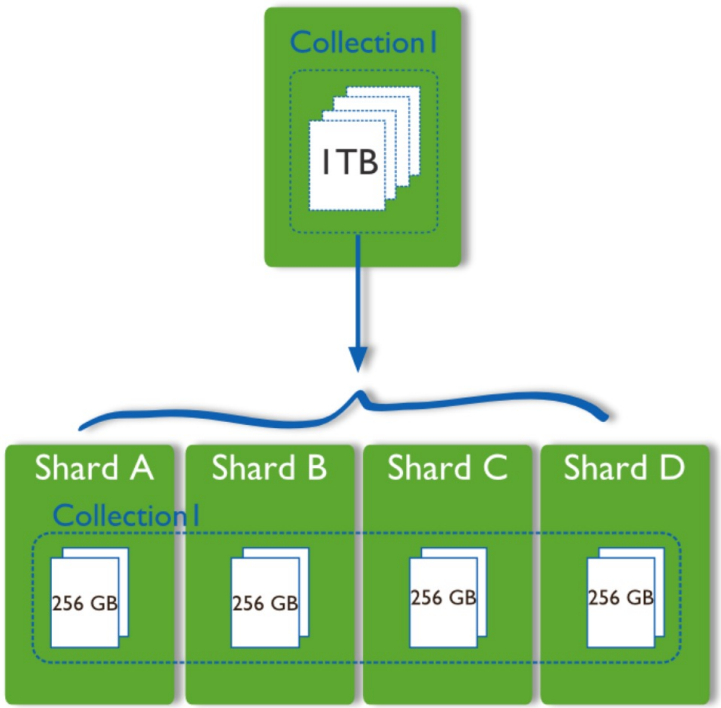


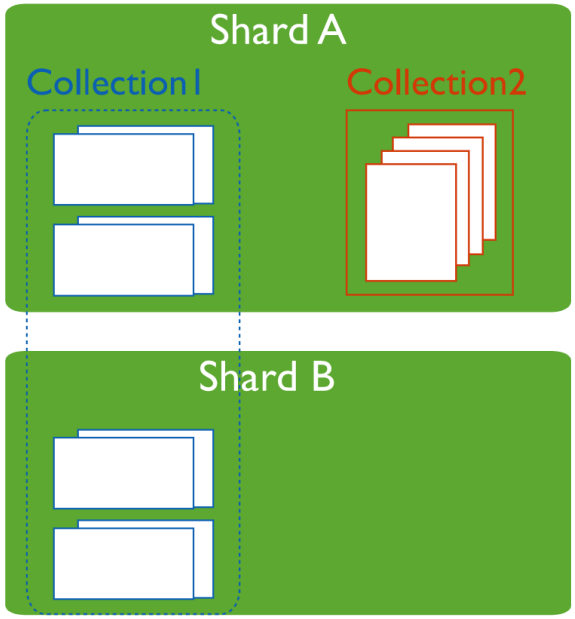
Evenly distributed across the cluster.



Each shard is replicated many times to form a replica set – several mongods would contain same shard.

Sharded and Non-Sharded Collections



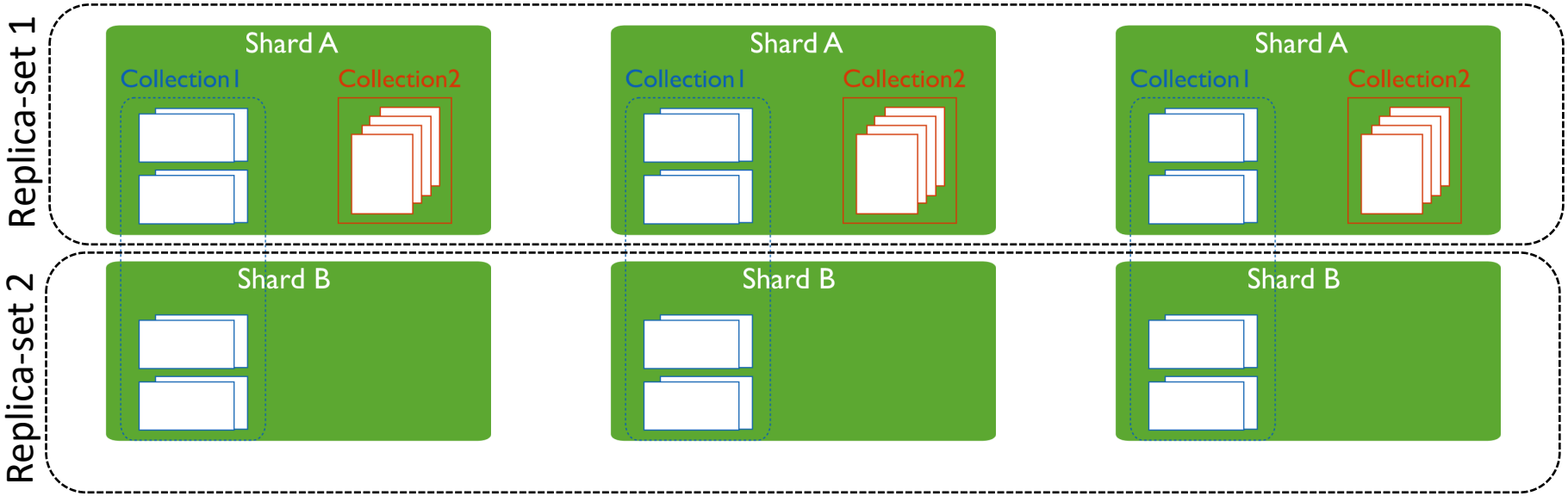


It is also possible in mongoDB to not shard a collection.

Non-shared ones are stored on a "primary shard" – each DB has its own Primary shard.

Refer: https://docs.mongodb.com/manual/sharding/

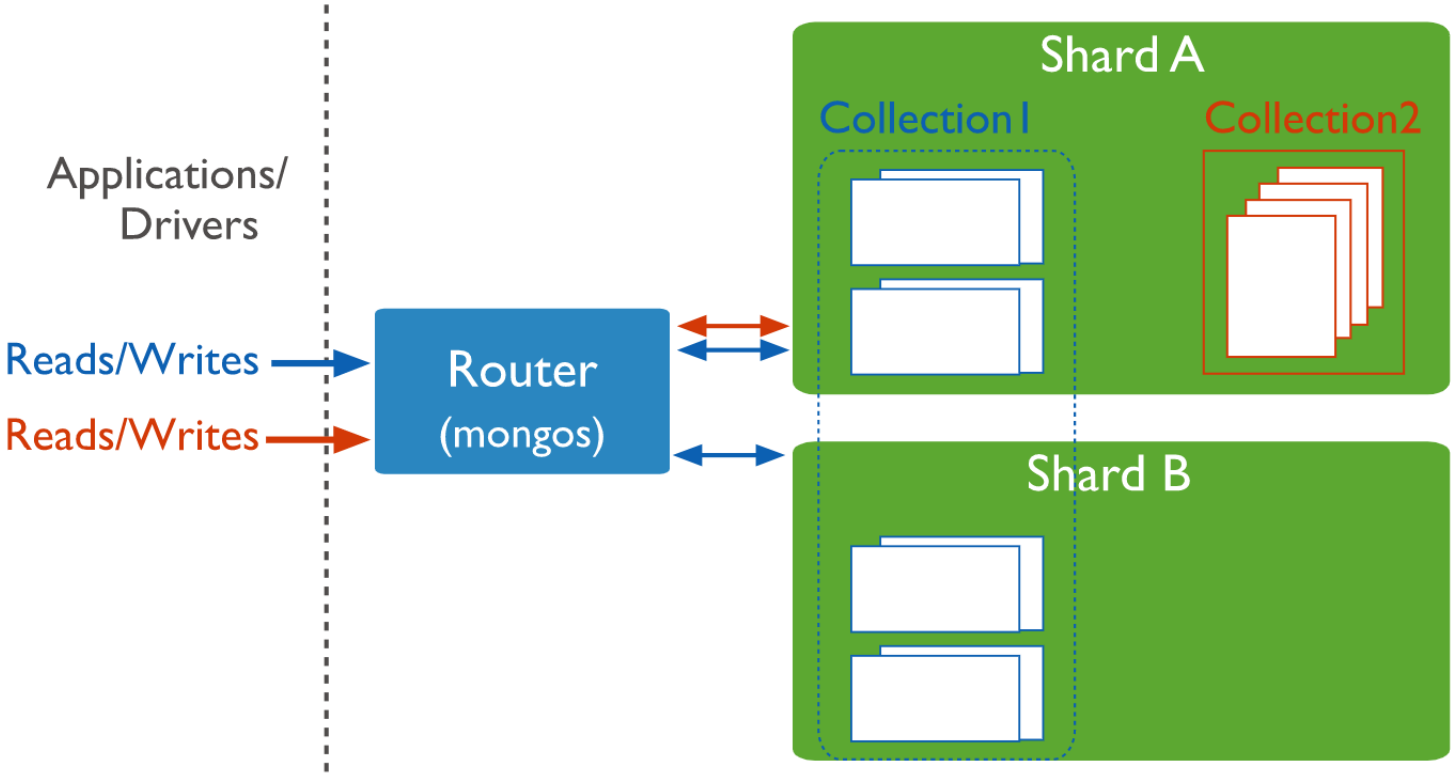
Sharded and Non-Sharded Collections



- There are mulitple replica sets each responsible for a particular shard.
- One mongod istance in a replica set act as a primary node.
- Here, the primary node of the Replica set-1 acts as the primary shard.

Refer: https://docs.mongodb.com/manual/sharding/

Connecting to Sharded Cluster



A client would first connect to a “mongos” to interact with any collection in the sharded cluster.

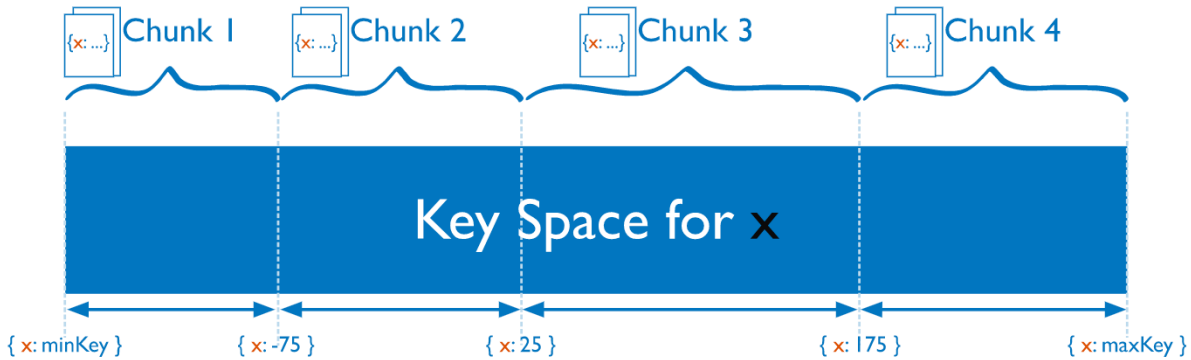
The mongos would pick the primary mongod on which the requested data resides.

Range- vs. Hash-Partitioning

Two main sharding strategies adopted by MongoDB.

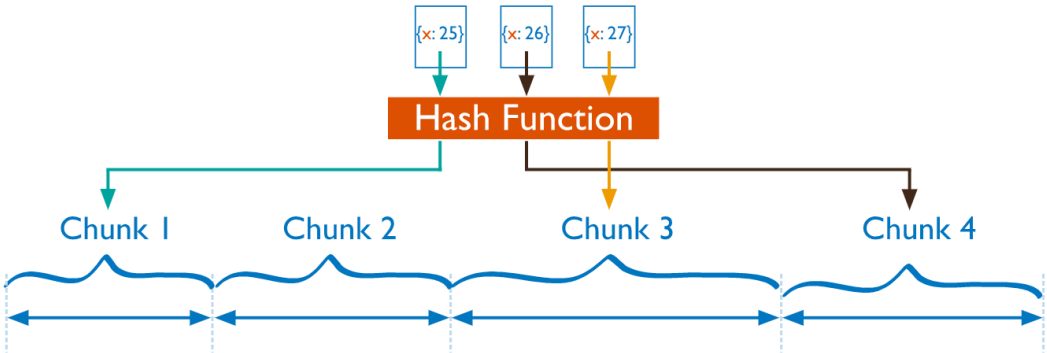
Option 1: **Range-Partitioning**

- Confirms to computing *x div m*, where m is the desired range of a partition.



Option 2: **Hash-Partitioning**

- Confirms to computing *x mod n*, where n is the desired number of partitions.



MongoDB CRUD Operations

CRUD Operations:

- CREATE
- READ
- UPDATE
- DELETE
- BULK WRITE

MongoDB CRUD Operations

CRUD Operations:

- **CREATE**
- READ
- UPDATE
- DELETE
- BULK WRITE

- Create or insert operations add new documents to a collection

- If the collection does not currently exist, insert operations will create the collection

Methods:

```
db.collection.insertOne()
db.collection.insertMany()
```

```
db.users.insertOne(          ← collection
  {
    name: "sue",             ← field: value
    age: 26,                 ← field: value   } document
    status: "pending"        ← field: value
  }
)
```

MongoDB CRUD Operations

CRUD Operations:

- **CREATE**
- READ
- UPDATE
- DELETE
- BULK WRITE

- Create or insert operations add new documents to a collection

- If the collection does not currently exist, insert operations will create the collection

Methods:
`db.collection.insertOne()`
`db.collection.insertMany()`

```
const doc1 = { "name": "basketball", "category": "sports", "quantity": 20, "reviews": [] };
const doc2 = { "name": "football", "category": "sports", "quantity": 30, "reviews": [] };

db.itemsCollection.insertMany([doc1, doc2])
```

MongoDB CRUD Operations

CRUD Operations:

- CREATE
- **READ**
- UPDATE
- DELETE
- BULK WRITE

- Retrieves documents from a collection

Methods:
`db.collection.find()`

```
db.users.find(                          ← collection
   { age: { $gt: 18 } },                ← query criteria
   { name: 1, address: 1 }              ← projection
).limit(5)                              ← cursor modifier
```

MongoDB CRUD Operations

CRUD Operations:

- CREATE
- READ
- **UPDATE**
- DELETE
- BULK WRITE

- Modify existing documents in a collection

Methods:

```
db.collection.updateOne()
db.collection.updateMany()
db.collection.replaceOne()
```

```
db.users.updateMany(                    ← collection
   { age: { $lt: 18 } },                ← update filter
   { $set: { status: "reject" } }       ← update action
)
```

- **ReplaceOne**: Replaces a single document within the collection based on the filter.

MongoDB CRUD Operations

CRUD Operations:

- CREATE
- READ
- UPDATE
- **DELETE**
- BULK WRITE

- Remove documents from a collection

Methods:

```
db.collection.deleteOne()
db.collection.deleteMany()
```

```
db.users.deleteMany(          ← collection
  { status: "reject" }        ← delete filter
)
```

MongoDB CRUD Operations

CRUD Operations:

- CREATE
- READ
- UPDATE
- DELETE
- **BULK WRITE**

- Provides the ability to perform bulk insert, update and remove operations

Methods:
`db.collection.bulkWrite()`

Supports the following write operations:
`insertOne`
`updateOne`
`updateMany`
`replaceOne`
`deleteOne`
`deleteMany`

```
db.characters.bulkWrite(
      [ { insertOne :
            { "document" :
               {"_id" : 4, "char" : "Dithras",
                 "class" : "barbarian", "lvl" : 4 }
            }
         },
         { insertOne :
            { "document" :
               {"_id" : 5, "char" : "Taeln",
                "class" : "fighter", "lvl" : 3 }
            }
         },
         { updateOne :
            {  "filter" : { "char" : "Eldon" },
               "update" : { $set : {
                     "status" : "Critical Injury" } }
            }
         },
         { deleteOne :
            { "filter" : { "char" : "Brisbane"} }
         },
         { replaceOne :
            {  "filter" : { "char" : "Meldane" },
               "replacement" : { "char" : "Tanys",
                          "class" : "oracle", "lvl" : 4 }
            }
         }] )
```

Bulkloading Large Files: `mongoimport`

- `mongoimport` supports JSON, CSV and TSV formats (all of which are ASCII-based) for bulk-loading large data volumes into a MongoDB collection

- From the Linux command line, use:

```
$mongoimport -- db <database-name> -u <groupname> -p <password>
            -- collection keywords
            -- fields docid, term, score
            -- type tsv –file <path-to-keywords.tsv>
```

- `mongoexport` is the counterpart of dumping a MongoDB collection into a file.

It writes (binary) BSON format back into ASCII/UTF format.